Indique con una cruz la opción correcta. Las preguntas de opción múltiple pueden tener varias respuestas correctas.

1. HTTP es un protocolo sin estado, lo que significa que mantiene información sobre la conexión entre transacciones.
   [ ] Verdadero. **[ ]** Falso.

2. ¿Cuál es el puerto estándar para el servicio HTTP?
   [ ] 443
   [ ] 21
   [ ] 80
   [ ] 22

3. ¿Qué indica un código de estado HTTP que comienza con el dígito 4 (4xx)?
   [ ] Éxito en la solicitud.
   [ ] Error en la parte del servidor.
   [ ] Error en la parte del cliente.
   [ ] Información únicamente.

4. ¿Qué métodos HTTP se definen habitualmente para una API REST?
   [ ] `GET` para recuperar información.
   [ ] `POST` para crear un nuevo recurso.
   [ ] `DELETE` para eliminar un recurso.
   [ ] `FETCH` para solicitar datos asíncronos.

5. ¿Qué elementos son considerados de "nivel de bloque" (Block-level elements) en HTML?
   [ ] `<p>`
   [ ] `<a>`
   [ ] `<h1> a <h6>`
   [ ] `<div>`

6. ¿Para qué sirve la declaración **`<!DOCTYPE html>`**?
   [ ] Para definir el título de la página.
   [ ] Para importar una hoja de estilos CSS.
   [ ] Para informar al navegador qué versión de HTML se está utilizando.
   [ ] Es una etiqueta obligatoria para definir el cuerpo del documento.

7. ¿Cuáles de las siguientes son ventajas de utilizar elementos semánticos en HTML5?
   [ ] Mejora la accesibilidad para lectores de pantalla.
   [ ] Facilita el mantenimiento del código.
   [ ] Ayuda a los motores de búsqueda a indexar mejor el contenido.
   [ ] Acelera la descarga de imágenes.

8. ¿Qué componentes integran el Box Model?
   [ ] Margin.
   [ ] Padding.
   [ ] Border.
   [ ] Content.

9. ¿Qué propiedad se utiliza para establecer un elemento como contenedor flex?
   [ ] `position: flex;`
   [ ] `flex-direction: row;`
   [ ] `display: flex;`
   [ ] `float: left;`

10. ¿Cuál es la diferencia entre **==** y **===** en JavaScript?
    [ ] No hay diferencia, son intercambiables.
    [ ] **==** es para asignación y **===** para comparación.
    [ ] **===** verifica igualdad sin realizar conversión automática de tipos.
    [ ] **==** es más rápido que **===**.

11. ¿Del lado del servidor, qué API de JavaScript permite acceder y modificar dinámicamente el contenido y la estructura de un documento HTML?
    [ ] HTTP.
    [ ] DOM (Document Object Model).
    [ ] JSON.
    [ ] V8.
    [ ] Ninguna de las anteriores.

12. ¿Qué método de selección en el DOM devuelve una lista de todos los elementos que coinciden con un selector CSS?
    [ ] **`getElementById`**.
    [ ] **`querySelector`**.
    [ ] **`querySelectorAll`**.
    [ ] **`getElementsByTagName`**.

13. ¿Qué es una función callback en JavaScript?
    [ ] Una función que se llama a sí misma.
    [ ] Una función que se pasa como argumento a otra función para ser ejecutada después.
    [ ] Una función que solo puede ser llamada por el servidor.
    [ ] Un método estático de la clase **`Object`**.

14. Al usar **`addEventListener`**, ¿qué se le pasa como segundo argumento?
    [ ] El nombre del elemento HTML.
    [ ] Un booleano que indica si el evento es asíncrono.
    [ ] El tipo de evento (como "click").
    [ ] Una función callback que se ejecutará cuando ocurra el evento.

15. ¿Qué diferencia al método **`map`** de otros métodos de iteración de arrays en JavaScript?
    [ ] Modifica el array original directamente para ahorrar memoria.
    [ ] Solo puede utilizarse con arrays de objetos.
    [ ] Crea un nuevo array del mismo tamaño con los valores transformados por una función.
    [ ] Reduce el array a un único valor numérico.

16. ¿Qué motor utiliza Node.js para compilar JavaScript a código máquina nativo?
    [ ] Libuv
    [ ] Nginx
    [ ] V8.
    [ ] Apache

17. JavaScript es un lenguaje single-threaded, pero puede delegar tareas asíncronas al entorno del navegador o Node.js.
    [ ] Verdadero. **[ ]** Falso.

18. Autenticación y Autorización son sinónimos que se refieren al proceso de verificar la identidad de un usuario.
    [ ] Verdadero. **[ ]** Falso.

19. ¿Qué tres partes componen un JSON Web Token (JWT)?
    [ ] Header (Encabezado).
    [ ] Payload (Carga útil).
    [ ] Signature (Firma).
    [ ] Protocolo de encriptación.

20. ¿Qué herramienta de Node.js se utiliza para gestionar dependencias e instalar paquetes?
    [ ] Express
    [ ] Morgan
    [ ] **`npm`**
    [ ] Middleware

21. ¿Qué palabra reservada se usa para pausar la ejecución de una función asíncrona hasta que una promesa se resuelva?
    [ ] **`await.`**
    [ ] **`then.`**
    [ ] **`async.`**
    [ ] **`resolve.`**

22. ¿Qué devuelve la función **`fetch()`**?
    [ ] El contenido del archivo JSON directamente.
    [ ] Un objeto de tipo **`Promise`**.
    [ ] Un objeto **`XMLHttpRequest`**.
    [ ] Una cadena de texto con formato HTML.

23. ¿Cuál es una limitación estricta al usar la palabra clave **`await`**?
    [ ] Solo puede usarse con el método **`POST`**.
    [ ] No permite el uso de bloques **`try/catch`**.
    [ ] Debe usarse dentro de una función que tenga el prefijo **`async`**.
    [ ] Requiere que el servidor tenga instalado Node.js.

24. ¿Para qué sirve el método **`response.json()`** en una solicitud **`fetch`**?
    [ ] Para enviar datos al servidor en formato JSON.
    [ ] Para convertir la respuesta en un objeto JavaScript real, devolviendo otra promesa.
    [ ] Para validar que el usuario está autenticado.
    [ ] Para cambiar el tipo MIME del documento HTML.

25. ¿Qué ventaja ofrece el uso de **`async`** y **`await`** sobre el uso de **`.then()`** y **`.catch()`**?
    [ ] Permite que el código sea más rápido en la ejecución.
    [ ] Simplifica e incluso elimina los anidamientos complicados de callbacks.
    [ ] No requiere conexión a Internet para funcionar.
    [ ] Solo funciona con bases de datos SQL.

26. ¿Qué indica el error de CORS al intentar usar **`fetch`**?
    [ ] Que el servidor ha caído por un error **`500`**.
    [ ] Que el JSON enviado tiene errores de sintaxis.
    [ ] Que el navegador bloqueó la solicitud porque el dominio del recurso es distinto al de la aplicación y no tiene los encabezados correctos.
    [ ] Que el usuario no ha ingresado sus credenciales básicas.

27. ¿Cuáles son las ventajas de la autenticación basada en tokens frente a las sesiones tradicionales?
- **[ ]** Es más escalable, ya que no requiere almacenar el estado en el servidor (stateless).
- **[ ]** Permite que aplicaciones móviles (fuera del navegador) utilicen la misma estrategia.
- **[ ]** Al no usar cookies por defecto, ayuda a mitigar ataques como CSRF.
- **[ ]** Los tokens nunca expiran, lo que facilita el acceso permanente.
- **[ ]** Todas las anteriores.
- **[ ]** Ninguna de las anteriores.

28. ¿Qué sucede con el token (accessToken) una vez que el servidor lo envía al cliente en una respuesta JSON?
- **[ ]** El navegador lo guarda automáticamente en una cookie de sesión cifrada.
- **[ ]** El JavaScript del cliente puede almacenarlo en **`sessionStorage`** o **`localStorage`** y adjuntarlo en futuros encabezados de solicitud.
- **[ ]** El token se destruye automáticamente tras el primer uso.
- **[ ]** El servidor debe guardarlo en su memoria para validarlo luego.

29. ¿Qué es un middleware?
- **[ ]** Un motor de base de datos ligero integrado en Node.
- **[ ]** Una ruta específica que solo acepta métodos **`POST`**.
- **[ ]** Una función que opera sobre la solicitud y la respuesta antes de que llegue al controlador.
- **[ ]** Un script que se ejecuta únicamente en el navegador del cliente.

30. Al programar un middleware personalizado, ¿qué es esencial para que la solicitud continúe su flujo?
- **[ ]** Ejecutar un comando **`return res.end()`**.
- **[ ]** No usar nunca funciones asíncronas.
- **[ ]** Realizar una llamada a la función **`next()`** para pasar al siguiente eslabón del pipeline.
- **[ ]** Reiniciar el servidor tras cada cambio en la lógica.

31. En una API REST protegida mediante JWT, ¿por qué no es necesario almacenar los access tokens en una base de datos para validar las solicitudes?
- **[ ]** Porque el token contiene la contraseña del usuario cifrada.
- **[ ]** Porque el token puede validarse verificando su firma y su fecha de expiración.
- **[ ]** Porque el navegador envía automáticamente el usuario y la contraseña en cada solicitud.
- **[ ]** Porque los tokens JWT son imposibles de falsificar aun sin firma.

32. Un DTO se utiliza para desacoplar el modelo interno de la base de datos de las respuestas que envía la API.
[ ] Verdadero. **[ ]** Falso.

33. ¿Cuál es la responsabilidad principal del Controlador?
- **[ ]** Gestionar la conexión directa y las transacciones con la base de datos SQL.
- **[ ]** Gestionar las solicitudes HTTP, procesar parámetros y enviar la respuesta con el código de estado adecuado.
- **[ ]** Definir los estilos CSS que se enviarán al cliente.
- **[ ]** Configurar los middlewares de seguridad globales como Helmet.

34. ¿Cuál es la principal diferencia de uso entre una URL absoluta y una relativa?
[ ] Una URL absoluta se necesita obligatoriamente cuando se vincula un documento con otros servidores.
[ ] Una URL relativa provee una referencia a otro documento basado en la localización del documento actual.
[ ] Las URLs absolutas solo funcionan dentro del mismo servidor local.
[ ] Las URLs relativas siempre deben incluir el esquema de protocolo (como **`http://`**).

35. ¿Qué beneficio directo tiene el usuario final con una implementación correcta de paginación?
[ ] Puede ver todos los datos de la empresa de una sola vez.
[ ] La seguridad de su contraseña mejora.
[ ] La interfaz carga más rápido al manejar volúmenes de datos reducidos y manejables.
[ ] El servidor web no necesita usar HTTPS.

36. En Express, el orden en el que se definen los middlewares de una ruta no afecta al funcionamiento de la aplicación.
[ ] Verdadero. **[ ]** Falso.

37. Completar la definición del formulario HTML:

```
<form action="/procesar-datos" method="POST">
  <label for="_______">Usuario:</label>
  <input type="____" name="usuario" id="usuario">

  <label for="_______">Contraseña:</label>
  <!-- Este tipo de input oculta los caracteres por seguridad -->
  <input type="________" name="contrasenia" id="contrasenia">

  <!-- Este tipo de button envía los datos-->
  <button type="______">Enviar</button>
</form>
```

38. Completar para definir una función asíncrona que consulte datos a un servidor web usando **`fetch`**:

```
____ function buscarAutos(url) {
  const respuesta = ____  _____(url);
  const datos = ____ respuesta.json();
  return datos;
}
```

39. Del lado del navegador completar para asociar el evento click a un botón:

```
const miBoton = ________.getElementById('btnParcial');

miBoton._________________('_____', function(_____) {
  _____.preventDefault();
  alert('¡Hiciste click en el botón!');
});
```

40. Completar para desestructurar el objeto persona obteniendo en variables separadas nombre y apellido:

```
const persona = { nombre: "Lionel Andrés", apellido: "Messi" };

const { ______, ________ } = persona;

console.log(______); // Resultado: "Lionel Andrés"
console.log(________); // Resultado: "Messi"
```